

Table of Contents

MY2029 Honda Civic - Comprehensive Vehicle Development Project	1
.....	2
.....	2
.....	2
.....	2
.....	2
.....	7
.....	7
.....	9
.....	9
.....	11
.....	11
.....	13
.....	13
.....	14
.....	15
.....	16
.....	16
.....	17
.....	17
.....	19
.....	19
.....	19
.....	19
.....	19
.....	25
.....	25
.....	27
.....	27
.....	29
.....	29
.....	30
.....	30

MY2029 Honda Civic - Comprehensive Vehicle Development Project

Single master script Corrected version using root Outport blocks + simOut.yout with fixed port order

```
clear; clc; close all; bdclose('all');
```

```
fprintf('\n=====\\n');
fprintf('MY2029 Honda Civic - Comprehensive Vehicle Development Project\\n');
fprintf('=====\\n\\n');
```

MY2029 Honda Civic - Comprehensive Vehicle Development Project

```

writetable(benchmarkTbl, fullfile(outDir, 'competitor_benchmark.csv'));

avgPrice = mean(Price_USD);
avgPower = mean(Power_hp);
avgFE     = mean(FuelEconomy_mpg);
avgMass   = mean(Mass_kg);
avgAccel  = mean(ZeroTo60_s);
avgCargo  = mean(Cargo_L);

marketAvgTbl = table(avgPrice, avgPower, avgFE, avgMass, avgAccel, avgCargo,
...
    'VariableNames', {'AvgPrice_USD', 'AvgPower_hp', 'AvgFuelEconomy_mpg', ...
        'AvgMass_kg', 'AvgZeroTo60_s', 'AvgCargo_L'});
disp(marketAvgTbl);

writetable(marketAvgTbl, fullfile(outDir, 'market_averages.xlsx'));

normPrice   = min(Price_USD)./Price_USD;
normPower   = Power_hp./max(Power_hp);
normFE      = FuelEconomy_mpg./max(FuelEconomy_mpg);
normMass    = min(Mass_kg)./Mass_kg;
normAccel   = min(ZeroTo60_s)./ZeroTo60_s;
normCargo   = Cargo_L./max(Cargo_L);
normSafety  = SafetyScore./max(SafetyScore);
normTech    = TechScore./max(TechScore);
normComfort = ComfortScore./max(ComfortScore);

weightsBenchmark = [0.12 0.14 0.16 0.08 0.12 0.08 0.12 0.10 0.08];

benchmarkIndex = ...
    weightsBenchmark(1)*normPrice + ...
    weightsBenchmark(2)*normPower + ...
    weightsBenchmark(3)*normFE + ...
    weightsBenchmark(4)*normMass + ...
    weightsBenchmark(5)*normAccel + ...
    weightsBenchmark(6)*normCargo + ...
    weightsBenchmark(7)*normSafety + ...
    weightsBenchmark(8)*normTech + ...
    weightsBenchmark(9)*normComfort;

benchmarkScoreTbl = table(Vehicle, benchmarkIndex, ...
    'VariableNames', {'Vehicle', 'BenchmarkIndex'});
benchmarkScoreTbl = sortrows(benchmarkScoreTbl, 'BenchmarkIndex', 'descend');
disp(benchmarkScoreTbl);

writetable(benchmarkScoreTbl, fullfile(outDir, 'benchmark_scores.xlsx'));

figure('Name', 'Competitor Benchmark - Price');
bar(categorical(Vehicle), Price_USD);
grid on;
ylabel('Price [USD]');
title('Competitor Price Comparison');

figure('Name', 'Competitor Benchmark - Power');

```

```

bar(categorical(Vehicle), Power_hp);
grid on;
ylabel('Power [hp]');
title('Competitor Power Comparison');

figure('Name','Competitor Benchmark - Fuel Economy');
bar(categorical(Vehicle), FuelEconomy_mpg);
grid on;
ylabel('Fuel Economy [mpg]');
title('Competitor Fuel Economy Comparison');

figure('Name','Competitor Benchmark - Overall Index');
bar(categorical(benchmarkScoreTbl.Vehicle),
benchmarkScoreTbl.BenchmarkIndex);
grid on;
ylabel('Benchmark Index [-]');
title('Overall Competitor Benchmark Index');

```

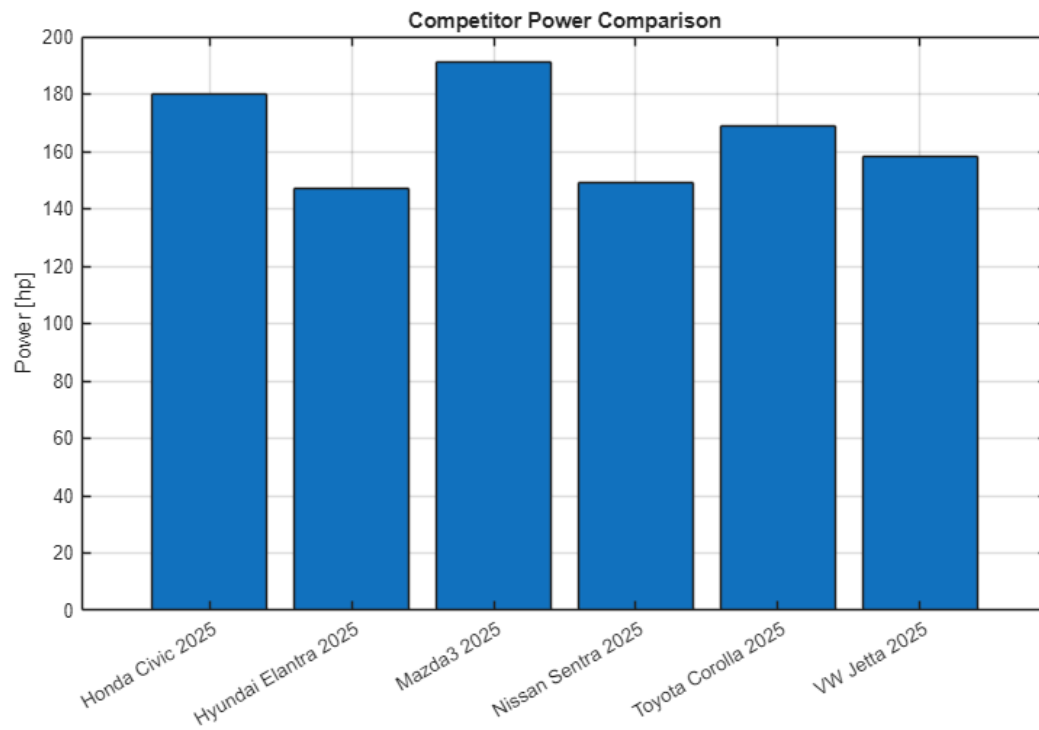
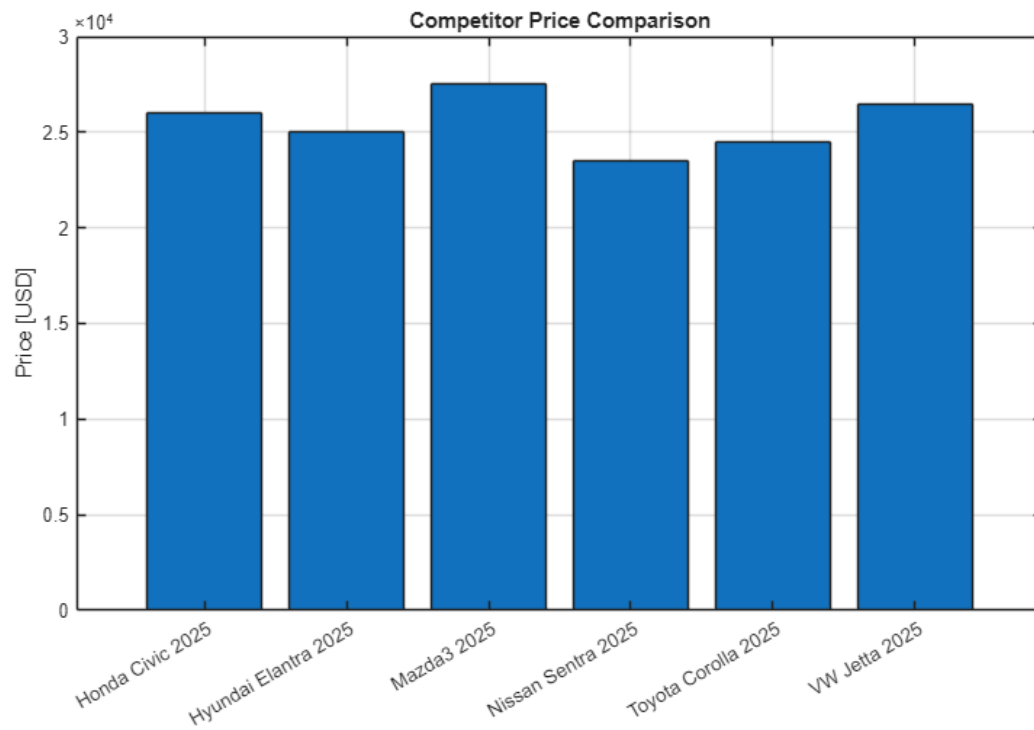
1) Running market analysis and competitor benchmarking...

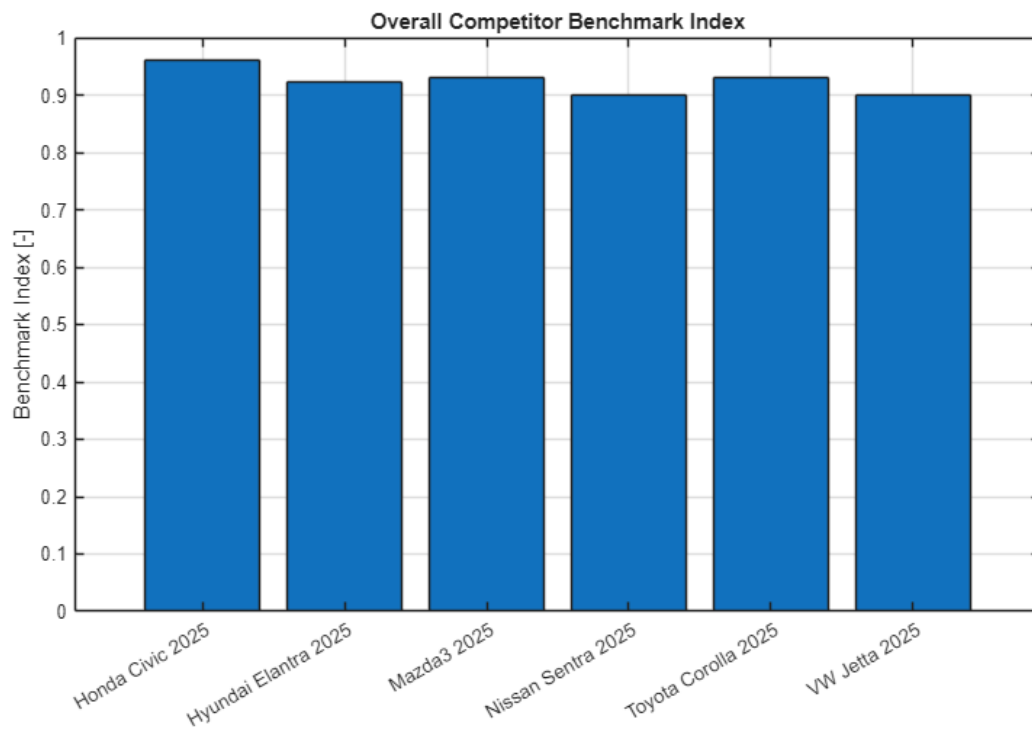
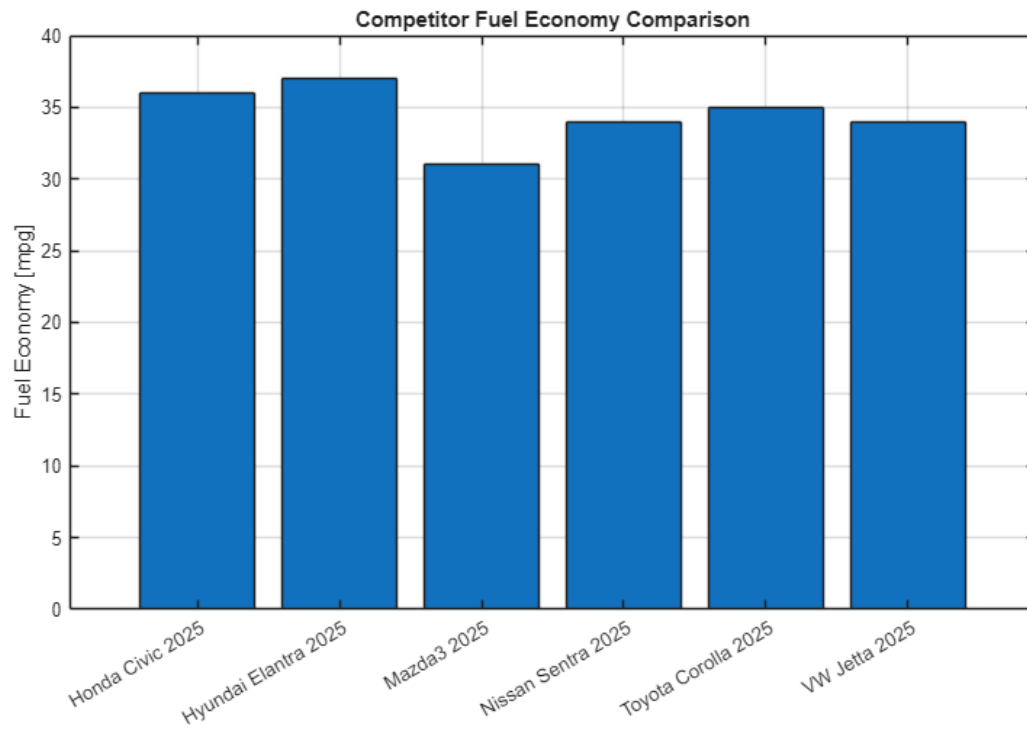
Vehicle			Price_USD	Power_hp	FuelEconomy_mpg
Mass_kg	ZeroTo60_s	Cargo_L	SafetyScore	TechScore	ComfortScore
{ 'Honda Civic 2025' }			26000	180	36
1360	7.8	419	9.2	8.8	8.6
{ 'Toyota Corolla 2025' }			24500	169	35
1330	8.2	371	9	8.3	8.4
{ 'Hyundai Elantra 2025' }			25000	147	37
1290	8.7	402	8.8	8.4	8.3
{ 'Mazda3 2025' }			27500	191	31
1410	7.1	374	8.9	8.1	8.7
{ 'Nissan Sentra 2025' }			23500	149	34
1320	8.9	405	8.5	7.8	8
{ 'VW Jetta 2025' }			26500	158	34
1390	8.4	399	8.7	8	8.2

AvgPrice_USD	AvgPower_hp	AvgFuelEconomy_mpg	AvgMass_kg
AvgZeroTo60_s	AvgCargo_L		
25500	165.67	34.5	1350
8.1833	395		

Vehicle	BenchmarkIndex
{ 'Honda Civic 2025' }	0.96027
{ 'Toyota Corolla 2025' }	0.93161
{ 'Mazda3 2025' }	0.92933
{ 'Hyundai Elantra 2025' }	0.92179
{ 'VW Jetta 2025' }	0.9009

{ 'Nissan Sentra 2025' } 0.90055





```

qfdPriority = Q' * importance;
qfdPriorityNorm = 100*qfdPriority/sum(qfdPriority);

qfdTbl = array2table(Q, ...
    'VariableNames', matlab.lang.makeValidName(engineeringChars), ...
    'RowNames', matlab.lang.makeValidName(customerNeeds));

priorityTbl = table(engineeringChars, qfdPriority, qfdPriorityNorm, ...
    'VariableNames',
    {'EngineeringCharacteristic', 'PriorityScore', 'PriorityPercent'});
priorityTbl = sortrows(priorityTbl, 'PriorityScore', 'descend');

disp(priorityTbl);

writetable(priorityTbl, fullfile(outDir, 'qfd_priority_results.xlsx'));
writetable(qfdTbl, fullfile(outDir, 'qfd_relationship_matrix.xlsx'),
    'WriteRowNames', true);

figure('Name', 'QFD Engineering Priority');
bar(categorical(priorityTbl.EngineeringCharacteristic),
    priorityTbl.PriorityScore);
grid on;
ylabel('Priority Score');
title('QFD Engineering Characteristic Priority');
xtickangle(35);

```

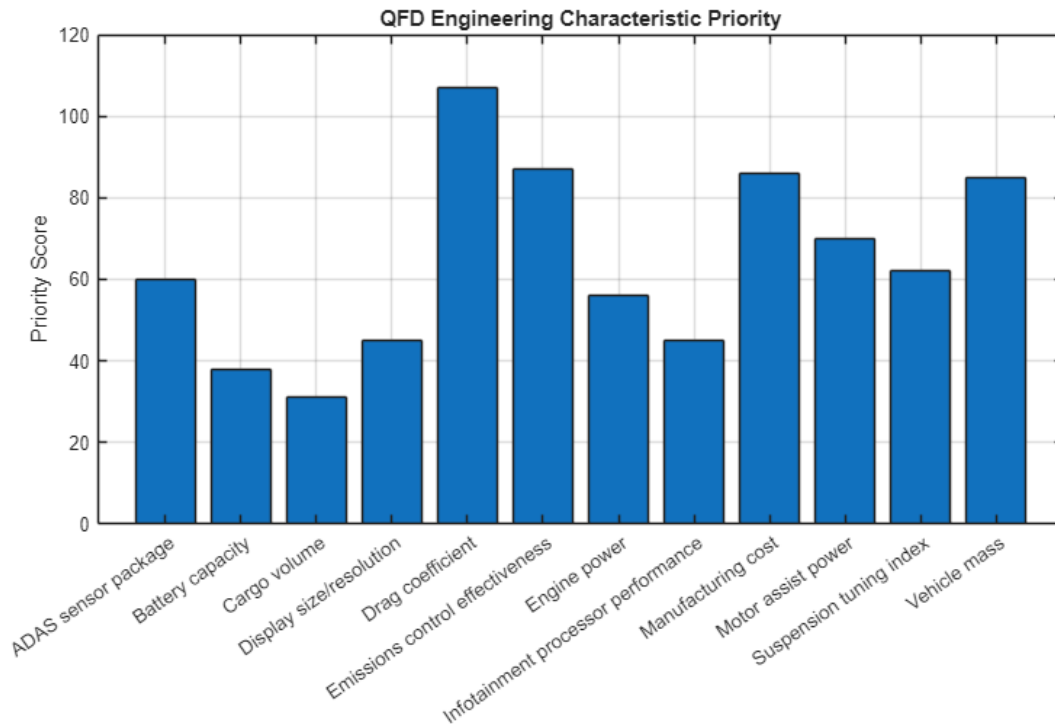
2) Running customer needs analysis and QFD...

<i>EngineeringCharacteristic</i>	<i>PriorityScore</i>
<i>PriorityPercent</i>	
{'Drag coefficient' }	107
13.86	
{'Emissions control effectiveness' }	87
11.269	
{'Manufacturing cost' }	86
11.14	
{'Vehicle mass' }	85
11.01	
{'Motor assist power' }	70
9.0674	
{'Suspension tuning index' }	62
8.0311	
{'ADAS sensor package' }	60
7.772	
{'Engine power' }	56
7.2539	
{'Infotainment processor performance' }	45
5.829	
{'Display size/resolution' }	45
5.829	
{'Battery capacity' }	38
4.9223	

```
{'Cargo volume'  
4.0155
```

```
}
```

```
31
```



SECTION 3: TARGET SPECIFICATIONS FOR MY2029 CIVIC

```
fprintf('3) Generating MY2029 Civic target specifications...\n');  
  
targetPrice_USD      = 27500;  
targetPower_hp       = 190;  
targetMotorPower_kW  = 18;  
targetBattery_kWh    = 1.8;  
targetFuelEconomy_mpg = 45;  
targetMass_kg        = 1425;  
targetZeroTo60_s     = 7.3;  
targetCd             = 0.25;  
targetCargo_L        = 430;  
targetSafetyRating   = 9.5;  
targetDisplay_in     = 12.3;  
targetEmissionsClass = "ULEV-focused mild hybrid target";
```

```

targetArchitecture      = "Front-wheel-drive mild hybrid compact sedan";

targetSpecNames = { ...
    'Target price [USD]'
    'Target power [hp]'
    'Motor assist power [kW]'
    'Battery energy [kWh]'
    'Target fuel economy [mpg]'
    'Target mass [kg]'
    'Target 0-60 mph [s]'
    'Target drag coefficient'
    'Target cargo volume [L]'
    'Target safety rating'
    'Display size [in]'
    'Emissions target'
    'Vehicle architecture' };

targetSpecValues = { ...
    targetPrice_USD
    targetPower_hp
    targetMotorPower_kW
    targetBattery_kWh
    targetFuelEconomy_mpg
    targetMass_kg
    targetZeroTo60_s
    targetCd
    targetCargo_L
    targetSafetyRating
    targetDisplay_in
    char(targetEmissionsClass)
    char(targetArchitecture) };

targetSpecTbl = table(targetSpecNames, targetSpecValues, ...
    'VariableNames', {'Specification','TargetValue'});
disp(targetSpecTbl);

writetable(targetSpecTbl, fullfile(outDir, 'target_specifications.xlsx'));

```

3) Generating MY2029 Civic target specifications...

Specification	
TargetValue	
{ 'Target price [USD]' }	
{ [27500] }	
{ 'Target power [hp]' }	
{ [190] }	
{ 'Motor assist power [kW]' }	
{ [18] }	
{ 'Battery energy [kWh]' }	
{ [1.8000] }	
{ 'Target fuel economy [mpg]' }	
{ [45] }	

```

        {'Target mass [kg]'          }
    {[                                1425]}
        {'Target 0-60 mph [s]'      }
    {[                                7.3000]}
        {'Target drag coefficient'   }
    {[                                0.2500]}
        {'Target cargo volume [L]'   }
    {[                                430]}
        {'Target safety rating'      }
    {[                                9.5000]}
        {'Display size [in]'         }
    {[                                12.3000]}
        {'Emissions target'          }    {'ULEV-focused mild hybrid
target'                                }
        {'Vehicle architecture'      }    {'Front-wheel-drive mild hybrid compact
sedan'                                }

```

```

=====

```

SECTION 4: REQUIREMENTS CASCADE / TRACEABILITY

```

=====

```

```

fprintf('4) Building requirements cascade and traceability...\n');

RequirementID = { ...
    'CR-01'; 'CR-02'; 'CR-03'; 'CR-04'; 'CR-05'; ...
    'ER-01'; 'ER-02'; 'ER-03'; 'ER-04'; 'ER-05'; ...
    'SR-01'; 'SR-02'; 'SR-03'; 'SR-04'; 'SR-05'};

RequirementType = { ...
    'Customer'; 'Customer'; 'Customer'; 'Customer'; 'Customer'; ...
    'Engineering'; 'Engineering'; 'Engineering'; 'Engineering';
'Engineering'; ...
    'Subsystem'; 'Subsystem'; 'Subsystem'; 'Subsystem'; 'Subsystem'};

Description = { ...
    'Vehicle shall be affordable in compact sedan market'
    'Vehicle shall provide high fuel efficiency'
    'Vehicle shall provide competitive acceleration'
    'Vehicle shall provide modern infotainment'
    'Vehicle shall ensure high occupant safety'
    'Vehicle mass shall be less than or equal to 1425 kg'
    'Combined peak power shall be approximately 190 hp'
    'Fuel economy target shall be at least 45 mpg'
    'Display size shall be at least 12.3 inches'
    'Vehicle drag coefficient shall be less than or equal to 0.25'

```

```

    'Battery subsystem shall support motor assist power target'
    'Powertrain controller shall manage engine-motor torque split'
    'Infotainment subsystem shall support responsive user interaction'
    'ADAS subsystem shall support basic active safety features'
    'Validation plan shall verify emissions and infotainment performance'};

ParentRequirement = { ...
    ''; ''; ''; ''; ...
    'CR-01'; 'CR-03'; 'CR-02'; 'CR-04'; 'CR-02'; ...
    'ER-02'; 'ER-02'; 'ER-04'; 'CR-05'; 'CR-05'};

VerificationMethod = { ...
    'Benchmarking'
    'Simulation/Test'
    'Simulation/Test'
    'Inspection/Test'
    'Test'
    'Calculation/Test'
    'Simulation/Test'
    'Simulation/Test'
    'Inspection/Test'
    'Simulation/Test'
    'Simulation/Test'
    'MIL/SIL/HIL'
    'Functional test'
    'Functional test'
    'Validation matrix'};

requirementsTbl = table(RequirementID, RequirementType, Description, ...
    ParentRequirement, VerificationMethod);
disp(requirementsTbl);

```

```

writetable(requirementsTbl, fullfile(outDir,
'requirements_traceability.xlsx'));

```

4) Building requirements cascade and traceability...

RequirementID	RequirementType	Description	ParentRequirement	VerificationMethod
{ 'CR-01' }	{ 'Customer' }	{ 'Vehicle shall be affordable in compact sedan market' }	{ 0×0 char }	
{ 'CR-02' }	{ 'Customer' }	{ 'Vehicle shall provide high fuel efficiency' }	{ 0×0 char }	{ 'Simulation/ Test' }
{ 'CR-03' }	{ 'Customer' }	{ 'Vehicle shall provide competitive acceleration' }	{ 0×0 char }	{ 'Simulation/ Test' }
{ 'CR-04' }	{ 'Customer' }	{ 'Vehicle shall provide modern infotainment' }	{ 0×0 char }	

```

{'Inspection/Test' }
    {'CR-05'}      {'Customer' }      {'Vehicle shall ensure
high occupant safety' }      {0x0 char}
{'Test' }
    {'ER-01'}      {'Engineering'}    {'Vehicle mass shall be less than
or equal to 1425 kg' }      {'CR-01' }      {'Calculation/
Test' }
    {'ER-02'}      {'Engineering'}    {'Combined peak power shall
be approximately 190 hp' }      {'CR-03' }
{'Simulation/Test' }
    {'ER-03'}      {'Engineering'}    {'Fuel economy target shall be at
least 45 mpg' }      {'CR-02' }      {'Simulation/
Test' }
    {'ER-04'}      {'Engineering'}    {'Display size shall be at least
12.3 inches' }      {'CR-04' }      {'Inspection/
Test' }
    {'ER-05'}      {'Engineering'}    {'Vehicle drag coefficient shall
be less than or equal to 0.25' }      {'CR-02' }      {'Simulation/
Test' }
    {'SR-01'}      {'Subsystem' }      {'Battery subsystem shall support
motor assist power target' }      {'ER-02' }      {'Simulation/
Test' }
    {'SR-02'}      {'Subsystem' }      {'Powertrain controller shall
manage engine-motor torque split' }      {'ER-02' }      {'MIL/SIL/
HIL' }
    {'SR-03'}      {'Subsystem' }      {'Infotainment subsystem
shall support responsive user interaction' }      {'ER-04' }
{'Functional test' }
    {'SR-04'}      {'Subsystem' }      {'ADAS subsystem shall support basic
active safety features' }      {'CR-05' }      {'Functional
test' }
    {'SR-05'}      {'Subsystem' }      {'Validation plan shall
verify emissions and infotainment performance' }      {'CR-05' }
{'Validation matrix'}

```

=====

=====

SECTION 5: CONCEPTUAL VEHICLE SIZING

=====

=====

```
fprintf('5) Running vehicle sizing calculations...\n');
```

```

m_vehicle = targetMass_kg;
rho_air   = 1.225;
A_frontal = 2.20;
Cd_target = targetCd;

```

```

#####

6) Generating financial development plan...\n');

CostElement = { ...
    'Base vehicle platform'
    'Mild hybrid motor system'
    'Battery pack'
    'Power electronics'
    'Infotainment hardware/software'
    'ADAS sensors/basic integration'
    'Validation and testing'
    'Manufacturing overhead'
    'Program management'
    'Supplier/tooling contingency'};

EstimatedCost_USD = [15000; 1200; 900; 600; 850; 700; 1200; 2300; 500; 750];

financialTbl = table(CostElement, EstimatedCost_USD);
financialTbl.TotalProgramVariableCost_USD = repmat(sum(EstimatedCost_USD),
height(financialTbl), 1);

disp(financialTbl);
writetable(financialTbl, fullfile(outDir, 'financial_plan.xlsx'));

figure('Name','Financial Cost Breakdown');
pie(EstimatedCost_USD, CostElement);
title('Estimated Cost Breakdown - MY2029 Civic Concept');

```

```

6) Generating financial development plan...
      CostElement      EstimatedCost_USD
TotalProgramVariableCost_USD

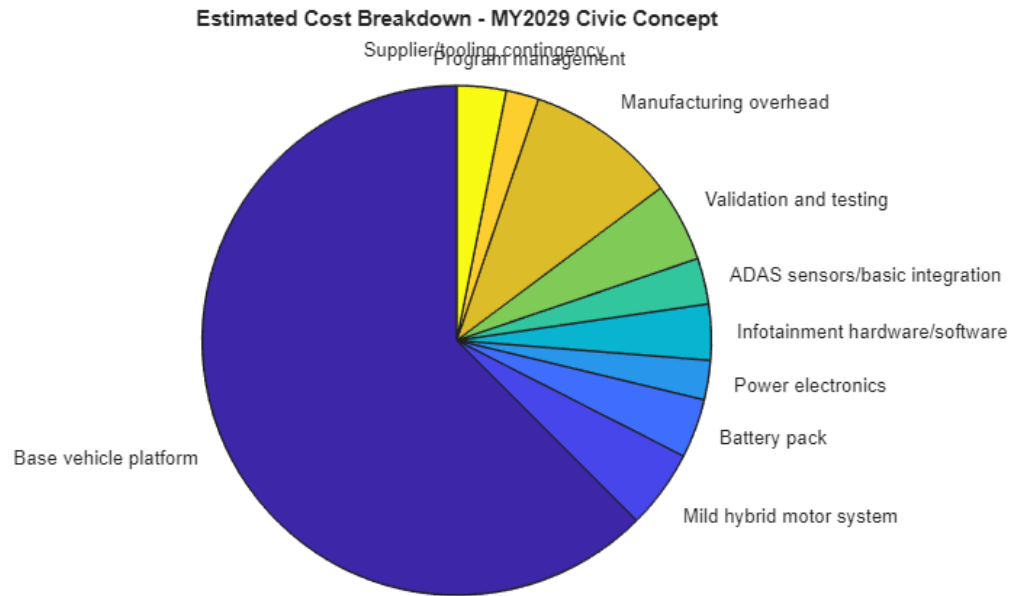
```

{ 'Base vehicle platform' }	15000
24000	
{ 'Mild hybrid motor system' }	1200
24000	
{ 'Battery pack' }	900
24000	
{ 'Power electronics' }	600
24000	
{ 'Infotainment hardware/software' }	850
24000	
{ 'ADAS sensors/basic integration' }	700
24000	
{ 'Validation and testing' }	1200
24000	
{ 'Manufacturing overhead' }	2300
24000	

```

    {'Program management'          }          500
24000
    {'Supplier/tooling contingency' }          750
24000

```



SECTION 7: TECHNOLOGY ROADMAP

```

fprintf('7) Creating technology roadmap...\n');

Year = [2026; 2027; 2028; 2029];
Milestone = { ...
    'Concept definition, benchmarking, QFD'
    'Subsystem sizing, architecture selection, controller development'
    'Prototype-level simulation validation, infotainment and emissions
planning'
    'Final validation readiness and MY2029 launch target'};

RoadmapFocus = { ...
    'Requirements capture'

```

```

        'Hybrid functional development'
        'Integrated validation'
        'Launch readiness'};

roadmapTbl = table(Year, Milestone, RoadmapFocus);
disp(roadmapTbl);
writetable(roadmapTbl, fullfile(outDir, 'technology_roadmap.xlsx'));

7) Creating technology roadmap...
    Year
Milestone
RoadmapFocus
-----

```

```

    2026    {'Concept definition, benchmarking,
QFD'                                     }    {'Requirements
capture'                                }
    2027    {'Subsystem sizing, architecture selection, controller
development'                            }    {'Hybrid functional development'}
    2028    {'Prototype-level simulation validation, infotainment and
emissions planning'}                    {'Integrated validation'            }
    2029    {'Final validation readiness and MY2029 launch
target'                                }    {'Launch readiness'                }

```

```

=====

```

```

=====

```

SECTION 8: VEHICLE VALIDATION PLAN

```

=====

```

```

=====

```

```

fprintf('8) Building vehicle validation plan...\n');

TestID =
{'VAL-01'; 'VAL-02'; 'VAL-03'; 'VAL-04'; 'VAL-05'; 'VAL-06'; 'VAL-07'; 'VAL-08'};
TestArea = { ...
    'Fuel economy'
    'Acceleration performance'
    'Top speed estimation'
    'Battery SOC behavior'
    'Emissions strategy'
    'Infotainment boot time'
    'Infotainment screen response'
    'Safety/ADAS functional check'};

TestMethod = { ...
    'Drive cycle simulation'

```

```

    '0-60 simulation'
    'Steady-state simulation'
    'Energy simulation'
    'Requirements/strategy verification'
    'Functional test'
    'Functional test'
    'Functional test'};

PassCriteria = { ...
    '>= 45 mpg equivalent target trend'
    '<= 7.3 s to 60 mph target trend'
    'Meets compact sedan expectation'
    'SOC remains within 0.4 to 0.8 operating band'
    'Low-emissions strategy implemented'
    '<= 3 s boot target'
    '<= 0.2 s touch response target'
    'All core functions verified'};

validationTbl = table(TestID, TestArea, TestMethod, PassCriteria);
disp(validationTbl);
writetable(validationTbl, fullfile(outDir, 'validation_plan.xlsx'));

8) Building vehicle validation plan...
    TestID          TestArea
TestMethod          PassCriteria

_____
_____

    {'VAL-01'}      {'Fuel economy'          }      {'Drive cycle
simulation'          }      {'>= 45 mpg equivalent target trend'          }
    {'VAL-02'}      {'Acceleration performance'    }      {'0-60
simulation'          }      {'<= 7.3 s to 60 mph target
trend'              }
    {'VAL-03'}      {'Top speed estimation'          }      {'Steady-state
simulation'          }      {'Meets compact sedan expectation'          }
    {'VAL-04'}      {'Battery SOC behavior'          }      {'Energy
simulation'          }      {'SOC remains within 0.4 to 0.8 operating
band'}
    {'VAL-05'}      {'Emissions strategy'          }      {'Requirements/
strategy verification'}      {'Low-emissions strategy implemented'          }
    {'VAL-06'}      {'Infotainment boot time'          }      {'Functional
test'                }      {'<= 3 s boot target'          }
    {'VAL-07'}      {'Infotainment screen response'}      {'Functional
test'                }      {'<= 0.2 s touch response target'          }
    {'VAL-08'}      {'Safety/ADAS functional check'}      {'Functional
test'                }      {'All core functions verified'          }

```

```

modelFile = fullfile(outDir, [modelName '.slx']);
if exist(modelFile, 'file')
    delete(modelFile);
end

new_system(modelName);
open_system(modelName);

simTime = 80;
set_param(modelName, 'StopTime', num2str(simTime));
set_param(modelName, 'Solver', 'ode45');
set_param(modelName, 'SaveOutput', 'on');
set_param(modelName, 'OutputSaveName', 'yout');
set_param(modelName, 'SaveFormat', 'Dataset');
set_param(modelName, 'SignalLogging', 'off');

% Vehicle / driveline parameters
m      = targetMass_kg;
Cd      = targetCd;
A      = A_frontal;
rho     = rho_air;
gearFD  = 4.60;      % slightly stronger final drive
etaDrv  = 0.94;      % driveline efficiency

% Tuned torque and power limits to better match target acceleration
TengMax = 300;      % Nm
TmotMax = 160;      % Nm
PmotMax = 30000;    % W
Vbatt   = 180;      % V
QbattAh = 20;       % Ah
QbattC  = QbattAh * 3600;
SOC0    = 0.60;

% More aggressive controller tuning
Kp      = 320;
Ki      = 45;

% Test profile
launchStartTime = 1.0;      % s
v60_ref        = 60 * 0.44704; % 26.8224 m/s
vHigh_ref_add   = 5.0;      % additional step later for higher-speed
behavior
highStepTime    = 35.0;      % s

x0 = 40;
y0 = 60;

% Reference profile:
% Step 1 -> real 0-60 launch request
% Step 2 -> later increase for top-speed-ish tracking
add_block('simulink/Sources/Step', [modelName '/Step_60mph'], ...
    'Time', num2str(launchStartTime), 'Before', '0', 'After',
    num2str(v60_ref), ...
    'Position', [x0 y0 x0+30 y0+30]);

```

```

add_block('simulink/Sources/Step', [modelName '/Step_HighSpeed'], ...
    'Time', num2str(highStepTime), 'Before', '0', 'After',
num2str(vHigh_ref_add), ...
    'Position', [x0 y0+60 x0+30 y0+90]);

add_block('simulink/Math Operations/Sum', [modelName '/Vref_Sum'], ...
    'Inputs', '++', ...
    'Position', [x0+70 y0+15 x0+100 y0+85]);

add_block('simulink/Math Operations/Sum', [modelName '/Speed_Error'], ...
    'Inputs', '+-', ...
    'Position', [x0+180 y0+15 x0+210 y0+85]);

add_block('simulink/Math Operations/Gain', [modelName '/Kp_Gain'], ...
    'Gain', num2str(Kp), ...
    'Position', [x0+260 y0 x0+315 y0+35]);

add_block('simulink/Continuous/Integrator', [modelName '/Int_Error'], ...
    'InitialCondition', '0', ...
    'Position', [x0+255 y0+65 x0+285 y0+95]);

add_block('simulink/Math Operations/Gain', [modelName '/Ki_Gain'], ...
    'Gain', num2str(Ki), ...
    'Position', [x0+325 y0+62 x0+380 y0+95]);

add_block('simulink/Math Operations/Sum', [modelName '/PI_Sum'], ...
    'Inputs', '++', ...
    'Position', [x0+425 y0+15 x0+455 y0+85]);

% Higher force demand ceiling for launch
add_block('simulink/Discontinuities/Saturation', [modelName '/Force_Sat'],
...
    'UpperLimit', '8000', 'LowerLimit', '-5000', ...
    'Position', [x0+500 y0+20 x0+560 y0+80]);

add_block('simulink/Math Operations/Gain', [modelName '/
Force_to_WheelTorque'], ...
    'Gain', num2str(rw), ...
    'Position', [x0+600 y0+20 x0+690 y0+80]);

% Torque split
add_block('simulink/Math Operations/Gain', [modelName '/Engine_Split'], ...
    'Gain', '0.65', ...
    'Position', [x0+740 y0 x0+805 y0+35]);

add_block('simulink/Math Operations/Gain', [modelName '/Motor_Split'], ...
    'Gain', '0.35', ...
    'Position', [x0+740 y0+65 x0+805 y0+100]);

add_block('simulink/Discontinuities/Saturation', [modelName '/
Engine_Torque_Sat'], ...
    'UpperLimit', num2str(TengMax), 'LowerLimit', '-60', ...
    'Position', [x0+840 y0 x0+915 y0+35]);

```

```

add_block('simulink/Discontinuities/Saturation', [modelName '/
Motor_Torque_Sat'], ...
    'UpperLimit', num2str(TmotMax), 'LowerLimit', num2str(-TmotMax), ...
    'Position', [x0+840 y0+65 x0+915 y0+100]);

add_block('simulink/Math Operations/Sum', [modelName '/Total_Drive_Torque'],
...
    'Inputs', '++', ...
    'Position', [x0+950 y0+20 x0+980 y0+90]);

add_block('simulink/Math Operations/Gain', [modelName '/Torque_to_Force'],
...
    'Gain', num2str((gearFD*etaDrv)/rw), ...
    'Position', [x0+1020 y0+15 x0+1110 y0+85]);

add_block('simulink/Math Operations/Product', [modelName '/Vsquare'], ...
    'Inputs', '**', ...
    'Position', [x0+1160 y0+130 x0+1215 y0+170]);

add_block('simulink/Math Operations/Gain', [modelName '/Aero_Gain'], ...
    'Gain', num2str(0.5*rho*Cd*A), ...
    'Position', [x0+1245 y0+130 x0+1335 y0+170]);

add_block('simulink/Sources/Constant', [modelName '/Rolling_Resistance'], ...
    'Value', num2str(Cr*m*g), ...
    'Position', [x0+1180 y0+215 x0+1260 y0+245]);

add_block('simulink/Math Operations/Sum', [modelName '/Resistance_Sum'], ...
    'Inputs', '++', ...
    'Position', [x0+1370 y0+145 x0+1400 y0+225]);

add_block('simulink/Math Operations/Sum', [modelName '/Net_Force'], ...
    'Inputs', '+-', ...
    'Position', [x0+1450 y0+20 x0+1480 y0+90]);

add_block('simulink/Math Operations/Gain', [modelName '/Inv_Mass'], ...
    'Gain', num2str(1/m), ...
    'Position', [x0+1510 y0+25 x0+1555 y0+80]);

add_block('simulink/Continuous/Integrator', [modelName '/Vehicle_Speed'], ...
    'InitialCondition', '0', ...
    'Position', [x0+1590 y0+25 x0+1625 y0+55]);

add_block('simulink/Continuous/Integrator', [modelName '/Vehicle_Distance'],
...
    'InitialCondition', '0', ...
    'Position', [x0+1660 y0+25 x0+1695 y0+55]);

add_block('simulink/Math Operations/Product', [modelName '/MotorPower_Raw'],
...
    'Inputs', '**', ...
    'Position', [x0+1020 y0+170 x0+1075 y0+210]);

```

```

add_block('simulink/Math Operations/Gain', [modelName '/MotorPower_Scale'],
...
    'Gain', num2str(1/rw), ...
    'Position', [x0+1115 y0+170 x0+1190 y0+210]);

add_block('simulink/Discontinuities/Saturation', [modelName '/
MotorPower_Sat'], ...
    'UpperLimit', num2str(PmotMax), 'LowerLimit', num2str(-PmotMax), ...
    'Position', [x0+1230 y0+170 x0+1310 y0+210]);

add_block('simulink/Math Operations/Gain', [modelName '/Power_to_Current'],
...
    'Gain', num2str(1/Vbatt), ...
    'Position', [x0+1345 y0+170 x0+1425 y0+210]);

add_block('simulink/Math Operations/Gain', [modelName '/Current_to_SOCdot'],
...
    'Gain', num2str(-1/QbattC), ...
    'Position', [x0+1455 y0+170 x0+1545 y0+210]);

add_block('simulink/Continuous/Integrator', [modelName '/SOC'], ...
    'InitialCondition', num2str(SOC0), ...
    'Position', [x0+1575 y0+170 x0+1610 y0+205]);

add_block('simulink/Discontinuities/Saturation', [modelName '/SOC_Sat'], ...
    'UpperLimit', '1', 'LowerLimit', '0', ...
    'Position', [x0+1640 y0+170 x0+1710 y0+205]);

add_block('simulink/Sinks/Scope', [modelName '/Scope_Speed'], ...
    'Position', [x0+1800 y0 x0+1860 y0+50]);

add_block('simulink/Sinks/Scope', [modelName '/Scope_SOC'], ...
    'Position', [x0+1800 y0+150 x0+1860 y0+200]);

add_block('simulink/Sinks/Scope', [modelName '/Scope_Torque'], ...
    'Position', [x0+1030 y0-85 x0+1090 y0-25]);

% Fixed output order
add_block('simulink/Ports & Subsystems/Out1', [modelName '/veh_speed'], ...
    'Port', '1', ...
    'Position', [x0+1740 y0+20 x0+1770 y0+40]);

add_block('simulink/Ports & Subsystems/Out1', [modelName '/soc_out'], ...
    'Port', '2', ...
    'Position', [x0+1740 y0+170 x0+1770 y0+190]);

add_block('simulink/Ports & Subsystems/Out1', [modelName '/veh_distance'],
...
    'Port', '3', ...
    'Position', [x0+1740 y0+70 x0+1770 y0+90]);

add_line(modelName, 'Step_60mph/1', 'Vref_Sum/1', 'autorouting', 'on');
add_line(modelName, 'Step_HighSpeed/1', 'Vref_Sum/2', 'autorouting', 'on');

```

```

add_line(modelName, 'Vref_Sum/1', 'Speed_Error/1', 'autorouting', 'on');
add_line(modelName, 'Vehicle_Speed/1', 'Speed_Error/2', 'autorouting', 'on');

add_line(modelName, 'Speed_Error/1', 'Kp_Gain/1', 'autorouting', 'on');
add_line(modelName, 'Speed_Error/1', 'Int_Error/1', 'autorouting', 'on');
add_line(modelName, 'Int_Error/1', 'Ki_Gain/1', 'autorouting', 'on');
add_line(modelName, 'Kp_Gain/1', 'PI_Sum/1', 'autorouting', 'on');
add_line(modelName, 'Ki_Gain/1', 'PI_Sum/2', 'autorouting', 'on');

add_line(modelName, 'PI_Sum/1', 'Force_Sat/1', 'autorouting', 'on');
add_line(modelName, 'Force_Sat/1', 'Force_to_WheelTorque/1', 'autorouting',
'on');

add_line(modelName, 'Force_to_WheelTorque/1', 'Engine_Split/1',
'autorouting', 'on');
add_line(modelName, 'Force_to_WheelTorque/1', 'Motor_Split/1',
'autorouting', 'on');
add_line(modelName, 'Engine_Split/1', 'Engine_Torque_Sat/1', 'autorouting',
'on');
add_line(modelName, 'Motor_Split/1', 'Motor_Torque_Sat/1', 'autorouting',
'on');

add_line(modelName, 'Engine_Torque_Sat/1', 'Total_Drive_Torque/1',
'autorouting', 'on');
add_line(modelName, 'Motor_Torque_Sat/1', 'Total_Drive_Torque/2',
'autorouting', 'on');
add_line(modelName, 'Total_Drive_Torque/1', 'Torque_to_Force/1',
'autorouting', 'on');

add_line(modelName, 'Vehicle_Speed/1', 'Vsquare/1', 'autorouting', 'on');
add_line(modelName, 'Vehicle_Speed/1', 'Vsquare/2', 'autorouting', 'on');
add_line(modelName, 'Vsquare/1', 'Aero_Gain/1', 'autorouting', 'on');
add_line(modelName, 'Aero_Gain/1', 'Resistance_Sum/1', 'autorouting', 'on');
add_line(modelName, 'Rolling_Resistance/1', 'Resistance_Sum/2',
'autorouting', 'on');

add_line(modelName, 'Torque_to_Force/1', 'Net_Force/1', 'autorouting', 'on');
add_line(modelName, 'Resistance_Sum/1', 'Net_Force/2', 'autorouting', 'on');
add_line(modelName, 'Net_Force/1', 'Inv_Mass/1', 'autorouting', 'on');
add_line(modelName, 'Inv_Mass/1', 'Vehicle_Speed/1', 'autorouting', 'on');
add_line(modelName, 'Vehicle_Speed/1', 'Vehicle_Distance/1', 'autorouting',
'on');

add_line(modelName, 'Motor_Torque_Sat/1', 'MotorPower_Raw/1', 'autorouting',
'on');
add_line(modelName, 'Vehicle_Speed/1', 'MotorPower_Raw/2', 'autorouting',
'on');
add_line(modelName, 'MotorPower_Raw/1', 'MotorPower_Scale/1', 'autorouting',
'on');
add_line(modelName, 'MotorPower_Scale/1', 'MotorPower_Sat/1', 'autorouting',
'on');
add_line(modelName, 'MotorPower_Sat/1', 'Power_to_Current/1', 'autorouting',
'on');
add_line(modelName, 'Power_to_Current/1', 'Current_to_SOCdot/1',

```

```

% Fixed mapping based on explicit output numbering:
% Port 1 -> veh_speed
% Port 2 -> soc_out
% Port 3 -> veh_distance
speedSig = getElement(yout, 1);
socSig    = getElement(yout, 2);
distSig   = getElement(yout, 3);

t_speed = speedSig.Values.Time;
v_speed = squeeze(speedSig.Values.Data);

t_soc = socSig.Values.Time;
v_soc = squeeze(socSig.Values.Data);

t_dist = distSig.Values.Time;
v_dist = squeeze(distSig.Values.Data);

maxSpeed_mps = max(v_speed);
maxSpeed_mph = maxSpeed_mps * 2.23694;
finalDistance_m = v_dist(end);

% True 0-60 metric: time from launch command to first crossing of 60 mph
launchStartTime = 1.0;
v60 = 60 * 0.44704;

idxAfterLaunch = find(t_speed >= launchStartTime);
t_speed_launch = t_speed(idxAfterLaunch);
v_speed_launch = v_speed(idxAfterLaunch);

if any(v_speed_launch >= v60)
    idx60 = find(v_speed_launch >= v60, 1, 'first');
    zeroTo60_est = t_speed_launch(idx60) - launchStartTime;
else
    zeroTo60_est = NaN;
end

simPerfTbl = table(maxSpeed_mps, maxSpeed_mph, finalDistance_m,
    zeroTo60_est, ...
    'VariableNames',
    {'MaxSpeed_mps', 'MaxSpeed_mph', 'Distance_m', 'ZeroTo60_Est_s'});
disp(simPerfTbl);

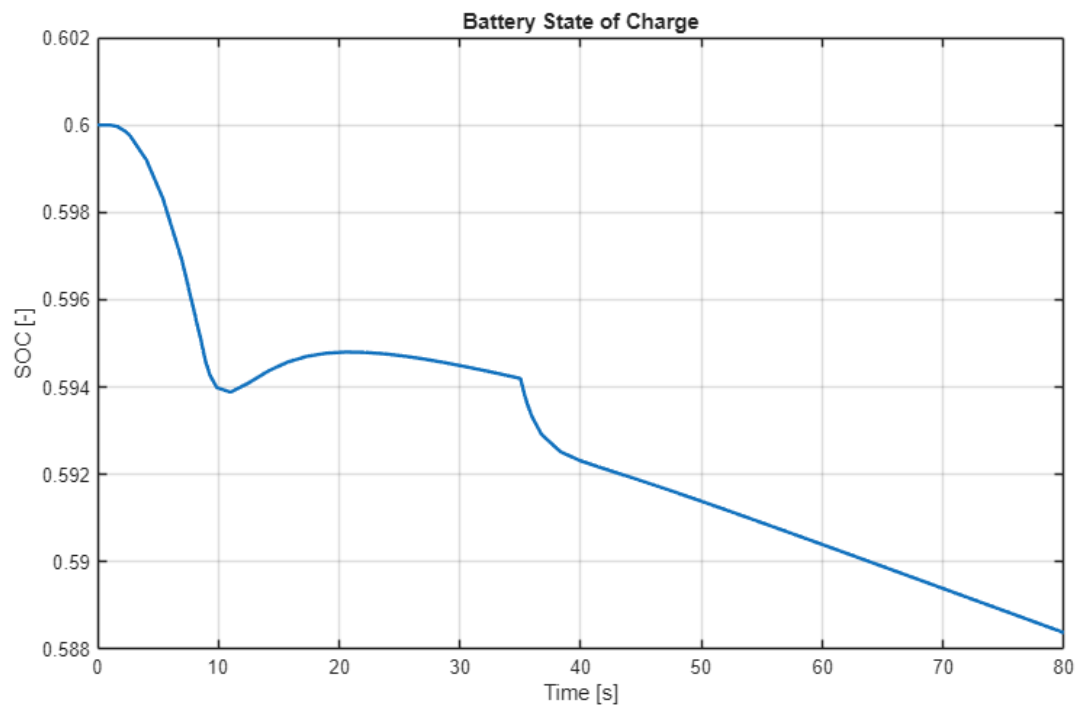
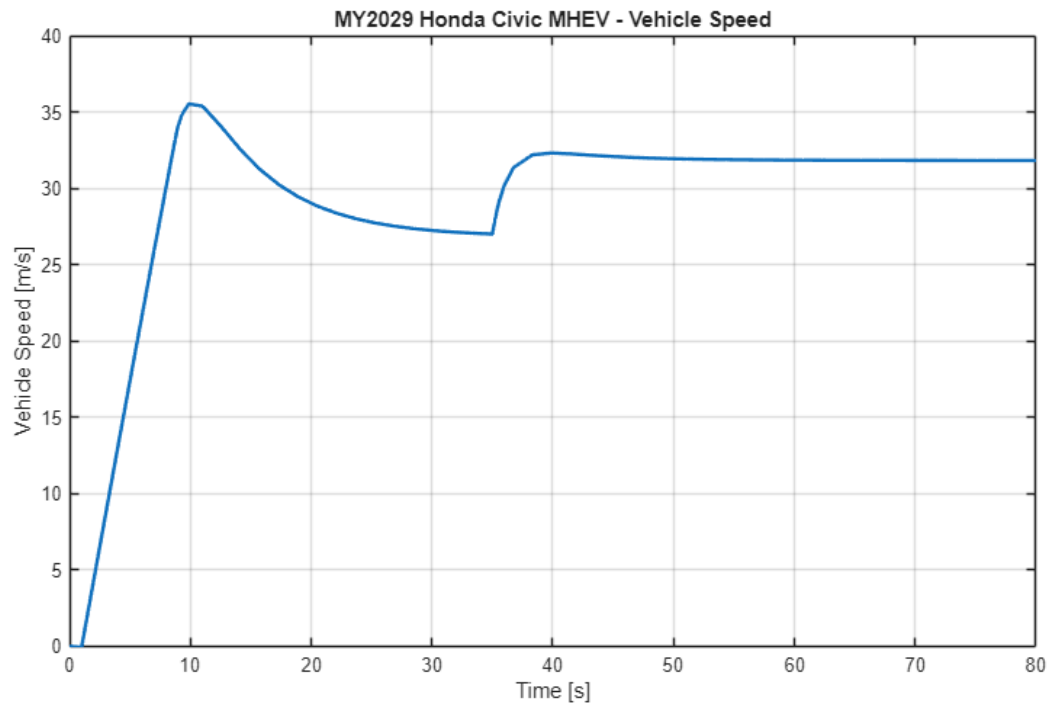
writetable(simPerfTbl, fullfile(outDir,
    'simulation_performance_results.xlsx'));

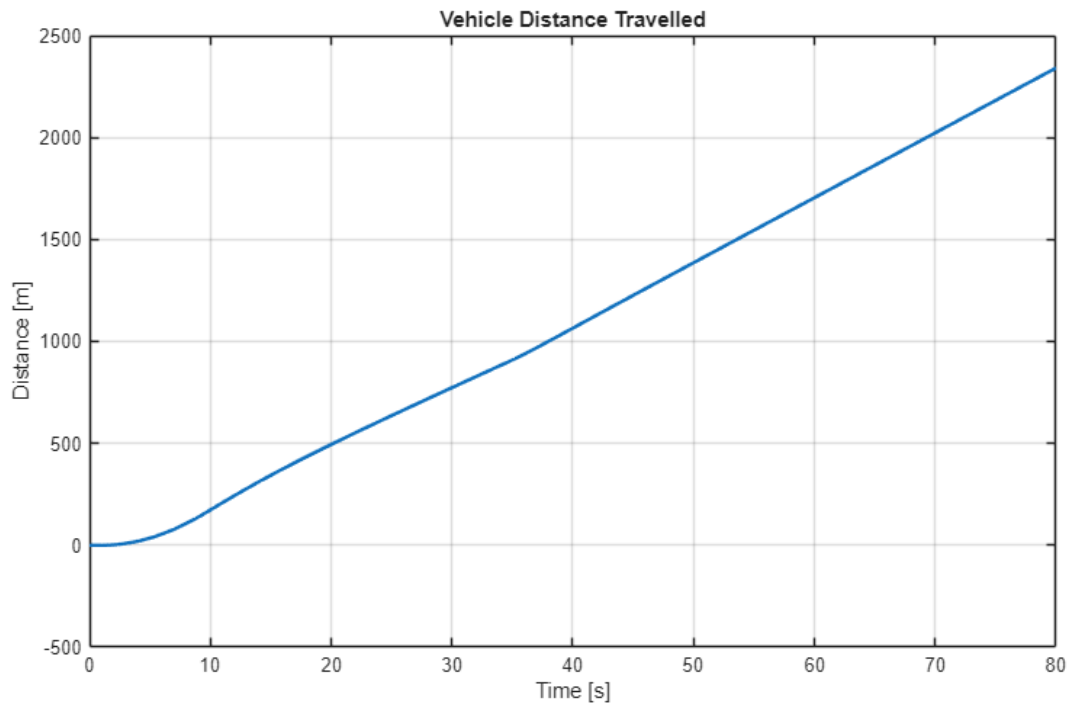
```

11) Running simulation...

Number of yout elements found: 3

<i>MaxSpeed_mps</i>	<i>MaxSpeed_mph</i>	<i>Distance_m</i>	<i>ZeroTo60_Est_s</i>
35.528	79.473	2341.3	7.5931





SECTION 13: SUMMARY REPORT TABLE

```
fprintf('13) Building executive summary...\n');
```

```
SummaryItem = { ...
    'Top competitor benchmark index vehicle'
    'Top engineering priority from QFD'
    'Target fuel economy [mpg]'
    'Target acceleration 0-60 [s]'
    'Target mass [kg]'
    'Estimated max simulation speed [mph]'
    'Estimated 0-60 from simulation [s]'
    'Final simulated distance [m]'};
```

```
SummaryValue = { ...
    benchmarkScoreTbl.Vehicle{1}
    priorityTbl.EngineeringCharacteristic{1}
    targetFuelEconomy_mpg
    targetZeroTo60_s
    targetMass_kg
```

```

        maxSpeed_mph
        zeroTo60_est
        finalDistance_m};

summaryTbl = table(SummaryItem, SummaryValue);
disp(summaryTbl);

writetable(summaryTbl, fullfile(outDir, 'project_summary.xlsx'));

13) Building executive summary...

```

<i>SummaryItem</i>	<i>SummaryValue</i>
{'Top competitor benchmark index vehicle'}	{'Honda Civic 2025'}
{'Top engineering priority from QFD' }	{'Drag coefficient'}
{'Target fuel economy [mpg]' }	{[45]}
{'Target acceleration 0-60 [s]' }	{[7.3000]}
{'Target mass [kg]' }	{[1425]}
{'Estimated max simulation speed [mph]' }	{[79.4730]}
{'Estimated 0-60 from simulation [s]' }	{[7.5931]}
{'Final simulated distance [m]' }	{[2.3413e+03]}

```

=====

=====

SECTION 14: SAVE MATLAB DATA

=====

=====

fprintf('14) Saving MATLAB workspace data...\n');

save(fullfile(outDir, 'my2029_civic_project_workspace.mat'));

fprintf('\n=====');
fprintf('PROJECT COMPLETE\n');
fprintf('All major files saved in:\n%s\n', outDir);
fprintf('=====');

```

```

14) Saving MATLAB workspace data...

=====
PROJECT COMPLETE
All major files saved in:
C:\Users\jaira\OneDrive\Documents\MATLAB_PROJECTS\honda civic
2029\MY2029_Civic_Project_Output\MY2029_Civic_Project_Output
=====

```

Published with MATLAB® R2025b